Lecture 6: Deep neural network-based representation learning, clustering

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 8, 2026

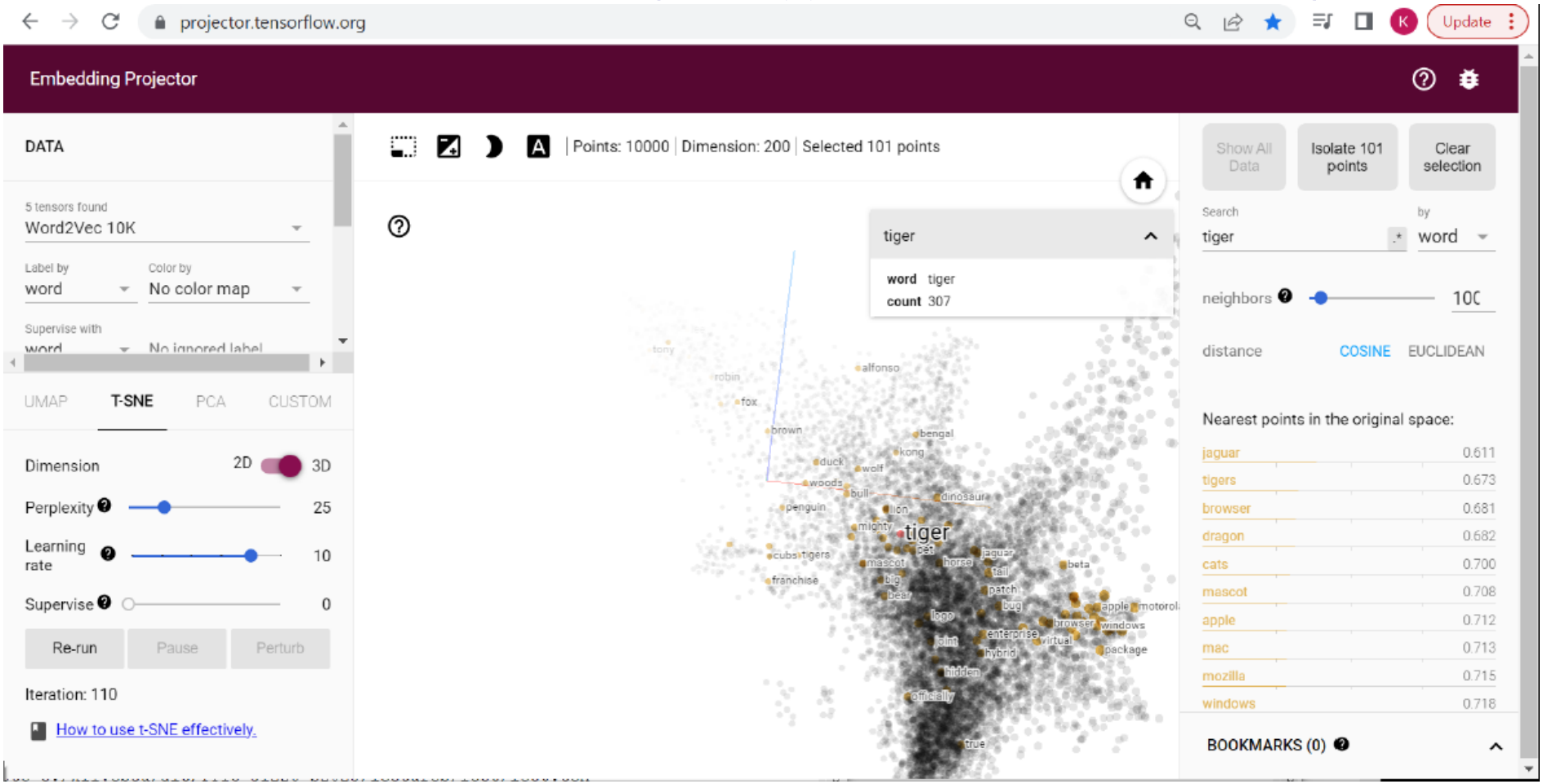
Announcement

All consent requests should have received a response

- If yours hasn't, please let me know

Follow-up from last time: t-SNE (and other) visualization tool

Tensorflow projector (https://projector.tensorflow.org)



Plan for today

- Variational autoencoders
- Start clustering

Unsupervised deep representation learning

Classes of deep rep learning techniques

- Use neural network as a form of out-of-sample extension for one of the methods we have discussed, e.g. parametric t-SNE (last time) [van der Maaten 2009]
- Autoencoders
- Probabilistic generative models that double as learned representations
  - Restricted Boltzmann machines, deep belief networks (we will probably not cover these)
  - Variational autoencoders (last time & today)
  - Future prediction models (including language models; later in the term)
- Self-supervised representation learning (later in the term)

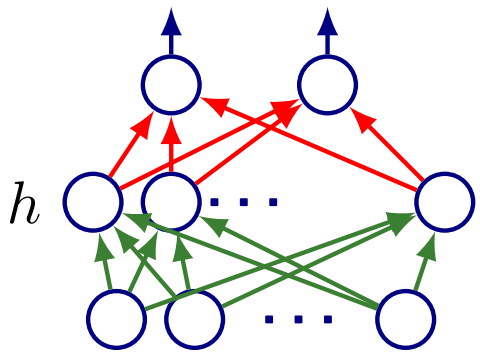
Deep learning: definition

- A predictor that uses a hierarchy of features of the input, typically (but not always) learned end-to-end jointly with the predictor.

$$f_y(\mathbf{x}) = F_L(F_{L-1}(F_{L-2}(\cdots F_1(\mathbf{x})\cdots)))$$

- Example: feedforward neural network (but the techniques we cover apply to any deep network)

- Feedforward operation, from input $\mathbf{x}$ to output $\hat{y}$:



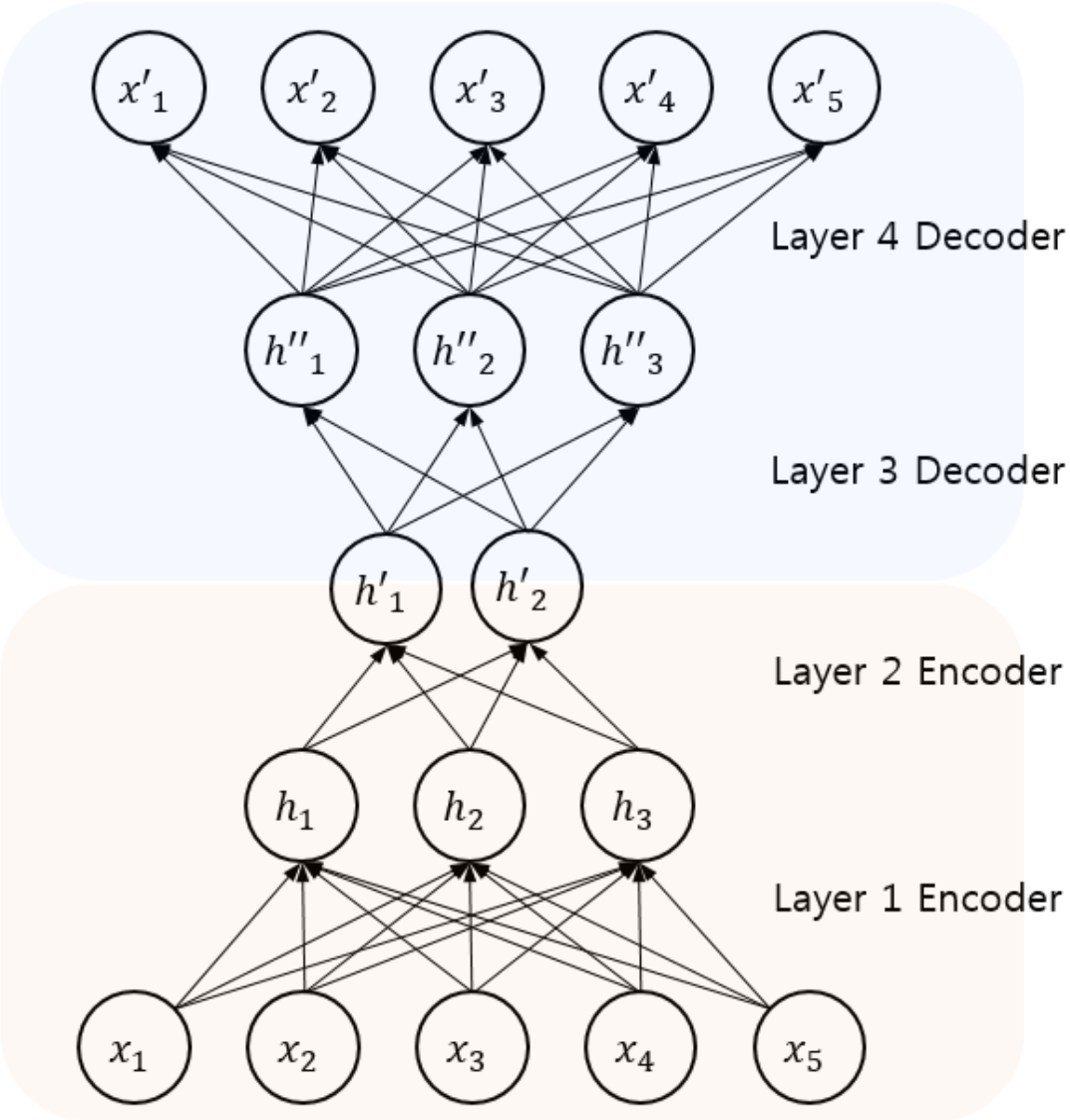
$$f_y(\mathbf{x}) = \sum_{j=1}^{m} w_{j,y}^{(2)} h\left(\sum_{i=1}^{d} w_{i,j}^{(1)} x_i + b_j^{(1)}\right) + b_y^{(2)}$$

- In matrix form:

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}_2 \cdot h\left(\mathbf{W}_1 \cdot \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2$$

Variational autoencoders

Reminder: Autoencoders



[Dürr 2016]

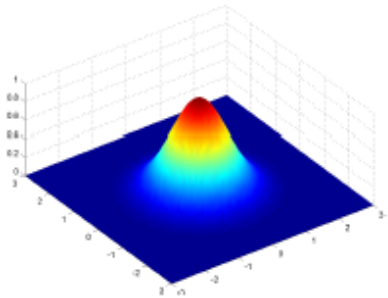
Variational autoencoders

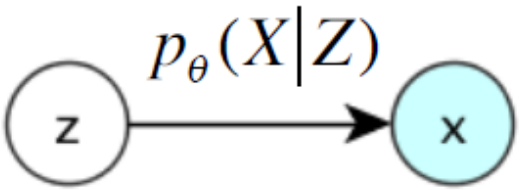
- More or less simultaneously proposed by Kingma & Welling (2013) and Rezende *et al.* (2014)
- Popular, fast, and relatively easy to train
- Generative model for a vector of random variables $\mathbf{x}$ assumed to be generated from a set of latent variables $\mathbf{z}$: $p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$
- For today, assume $\mathbf{x}, \mathbf{z}$ are both continuous
- Initially used for generation, but has also been successful for representation learning

VAE generation model (decoder)

$z \sim p(z)$ multivariate Gaussian

$x|z \sim p_\theta(x|z)$



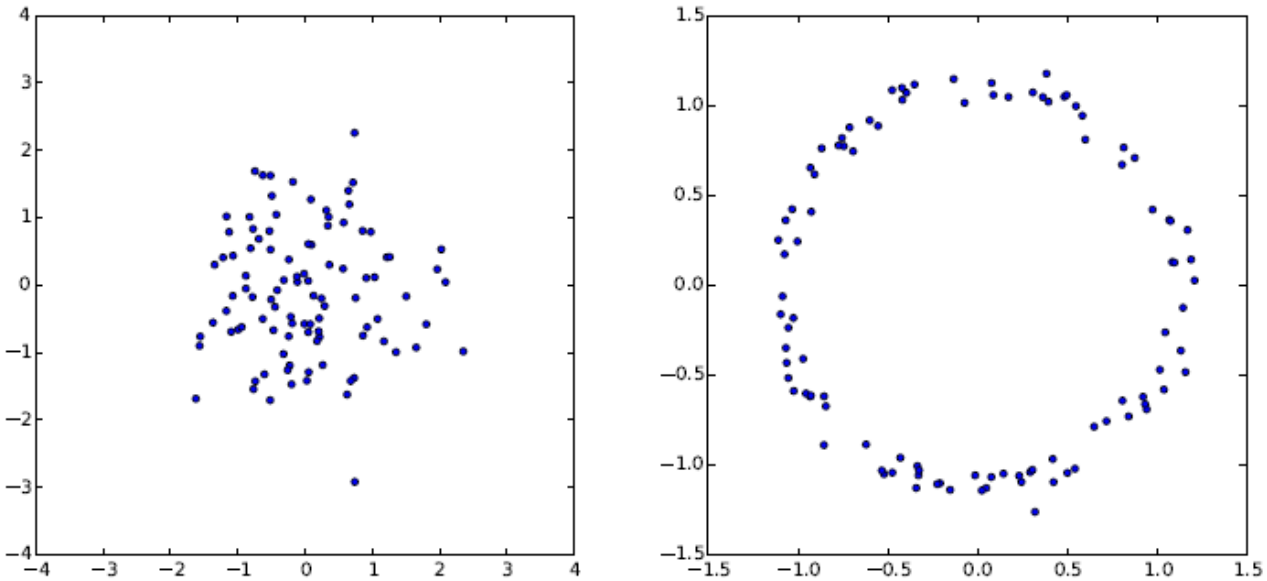




[Dürr 2016]

VAE generation model (decoder)

Key insight: Any d-dimensional distribution can be generated by taking d normally distributed variables and mapping them through some appropriate (possibly very complicated) function

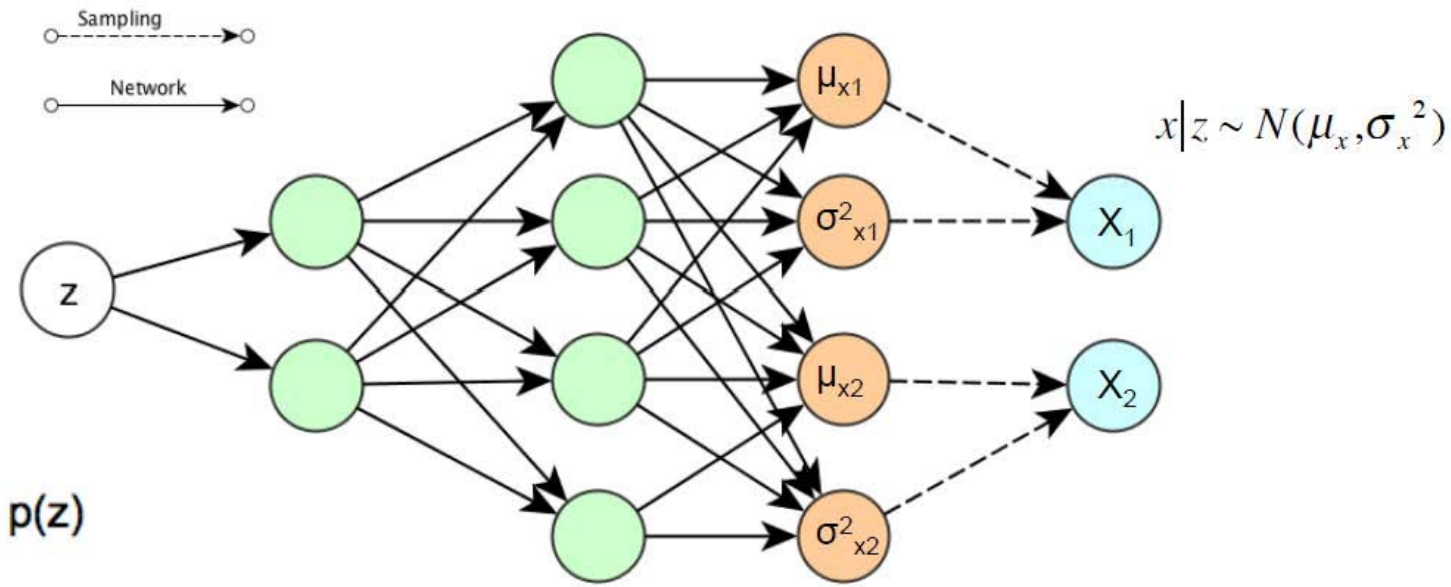


[Doersch 2016]

Here $x = g(z) = z/10 + z/||z||$

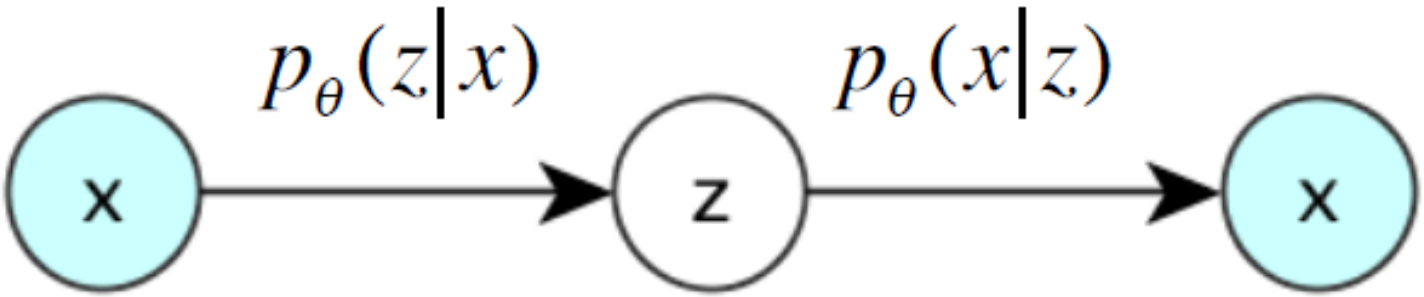
VAE neural decoder

- Assume w.l.o.g. that latent variable $\mathbf{z}$ is 1-dimensional, $\mathbf{x}$ is 2-dimensional
- For now, assume that $\mathbf{z}$ is Gaussian, $\mathbf{x}$ is diagonal Gaussian conditioned on $\mathbf{z}$



[Dürr 2016]

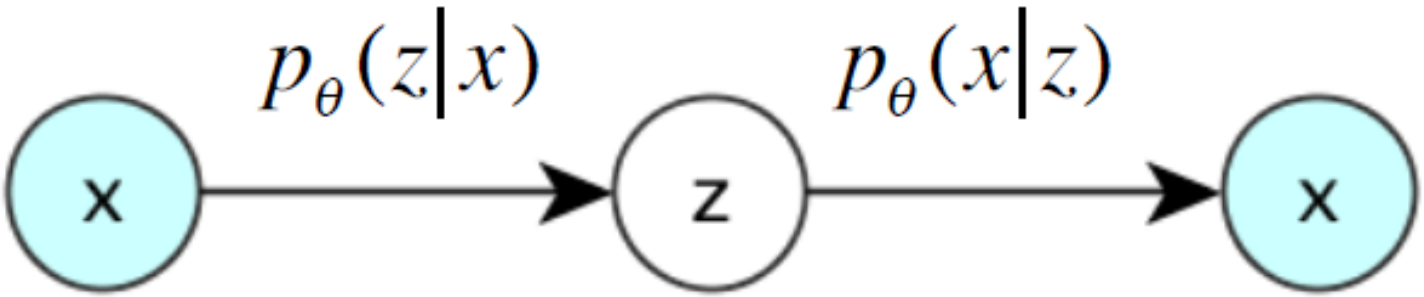
VAE encoder-decoder model



[Dürr 2016]

- But $p(\mathbf{z}|\mathbf{x})$ can be very costly to estimate
- We will approximate it with another neural function $q(\mathbf{z}|\mathbf{x})$
- That function $q(\mathbf{z}|\mathbf{x})$ is the encoder

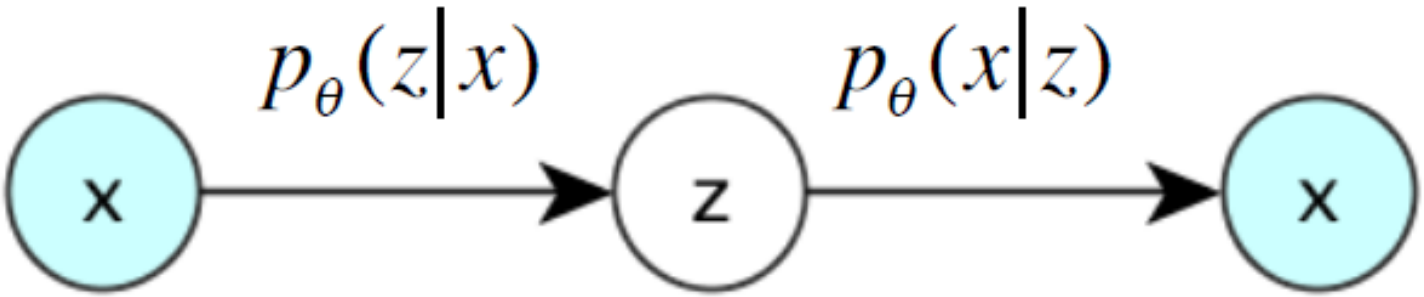
VAE encoder-decoder model



[Dürr 2016]

- But $p(\mathbf{z}|\mathbf{x})$ can be very costly to estimate
- We will approximate it with another neural function $q(\mathbf{z}|\mathbf{x})$
- That function $q(\mathbf{z}|\mathbf{x})$ is the encoder
- We can think of a VAE as a data generator...
  - Then we only need the decoder; encoder just helps us train the decoder

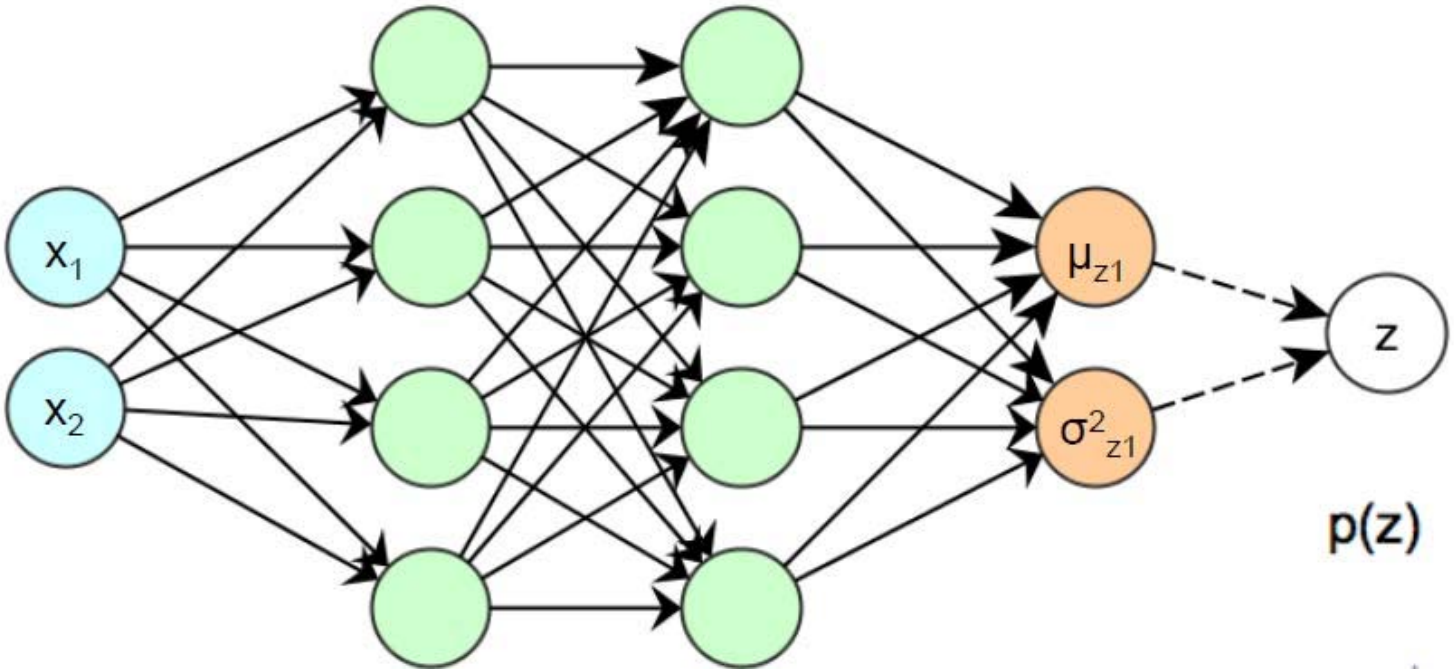
VAE encoder-decoder model



[Dürr 2016]

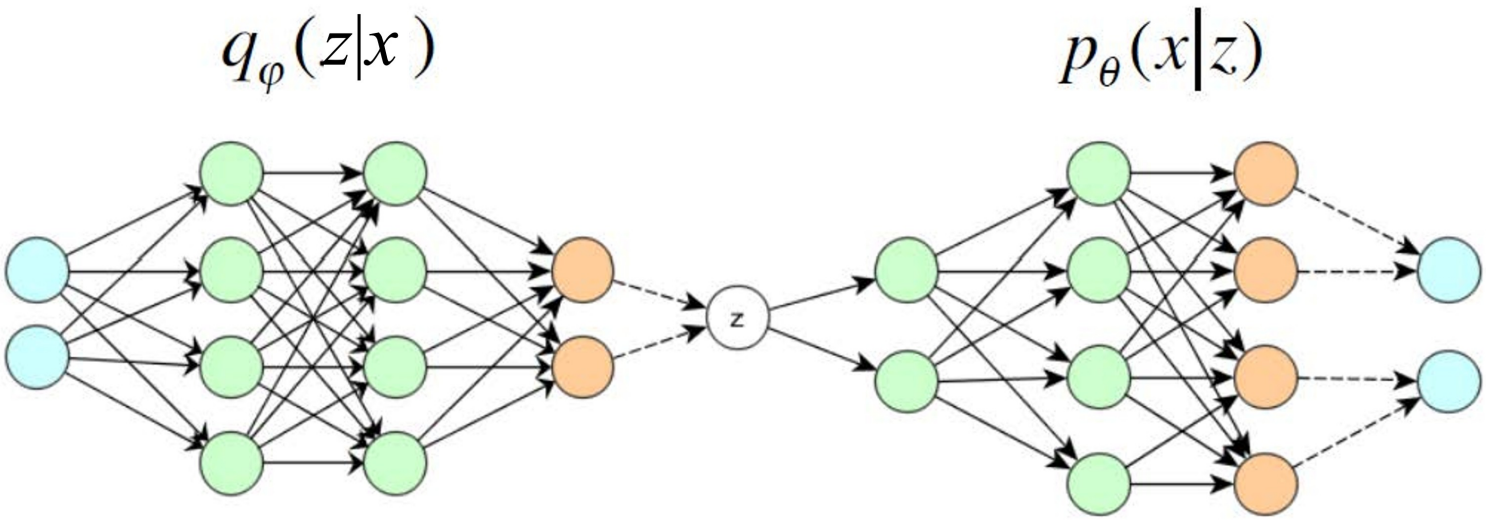
- But $p(\mathbf{z}|\mathbf{x})$ can be very costly to estimate
- We will approximate it with another neural function $q(\mathbf{z}|\mathbf{x})$
- That function $q(\mathbf{z}|\mathbf{x})$ is the encoder
- We can think of a VAE as a data generator...
  - Then we only need the decoder; encoder just helps us train the decoder
- Or as a representation model...
  - Then we only need the encoder; decoder just helps us train the encoder

VAE neural encoder



[Dürr 2016]

Complete VAE



[Dürr 2016]

Both a learned representation and a generative model!

- Where is the learned representation?
- How do we generate from this model?

Learning: Maximizing (a lower bound on the) likelihood

$$L = \log p(x)$$

Learning: Maximizing (a lower bound on the) likelihood

$$
\begin{aligned}
L &= \log p(x) \\
\text{multiply by 1...} \quad &= \sum_z q(z|x) \log p(x) \\
&= \sum_z q(z|x) \log \left(\frac{p(z,x)}{p(z|x)} \right)
\end{aligned}
$$

Learning: Maximizing (a lower bound on the) likelihood

$$
\begin{aligned}
L &= \log p(x) \\
\text{multiply by 1...} \quad &= \sum_z q(z|x) \log p(x) \\
&= \sum_z q(z|x) \log \left(\frac{p(z,x)}{p(z|x)} \right) \\
\text{multiply by 1...} \quad &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \frac{q(z|x)}{p(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) + \sum_z q(z|x) \log \left(\frac{q(z|x)}{p(z|x)} \right)
\end{aligned}
$$

Learning: Maximizing (a lower bound on the) likelihood

$$
\begin{aligned}
L &= \log p(x) \\
\text{multiply by 1...} \quad &= \sum_z q(z|x) \log p(x) \\
&= \sum_z q(z|x) \log \left(\frac{p(z,x)}{p(z|x)} \right) \\
\text{multiply by 1...} \quad &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \frac{q(z|x)}{p(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) + \sum_z q(z|x) \log \left(\frac{q(z|x)}{p(z|x)} \right) \\
&= L^v + D_{KL}(q(z|x)||p(z|x))
\end{aligned}
$$

Learning: Maximizing (a lower bound on the) likelihood

$$
\begin{aligned}
L &= \log p(x) \\
\text{multiply by 1...} \quad &= \sum_z q(z|x) \log p(x) \\
&= \sum_z q(z|x) \log \left(\frac{p(z,x)}{p(z|x)} \right) \\
\text{multiply by 1...} \quad &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \frac{q(z|x)}{p(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) + \sum_z q(z|x) \log \left(\frac{q(z|x)}{p(z|x)} \right) \\
&= L^v + D_{KL}(q(z|x)||p(z|x)) \\
&\geq L^v \quad \text{(the "evidence lower bound" (ELBO))}
\end{aligned}
$$

Rewriting the lower bound...

$$
\begin{aligned}
L^v &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(x|z)p(z)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z)}{q(z|x)} \right) + \sum_z q(z|x) \log p(x|z)
\end{aligned}
$$

Rewriting the lower bound...

$$
\begin{aligned}
L^v &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(x|z)p(z)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z)}{q(z|x)} \right) + \sum_z q(z|x) \log p(x|z) \\
&= -D_{KL}\left(q(z|x)||p(z)\right) + E_{q(z|x)}\left(\log p(x|z)\right)
\end{aligned}
$$

Rewriting the lower bound...

$$
\begin{aligned}
L^v &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(x|z)p(z)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z)}{q(z|x)} \right) + \sum_z q(z|x) \log p(x|z) \\
&= -D_{KL}\left(q(z|x)||p(z)\right) + E_{q(z|x)}\left(\log p(x|z)\right) \\
\text{for } x_i\ldots \quad &= -D_{KL}\left(q(z|x_i)||p(z)\right) + E_{q(z|x_i)}\left(\log p(x_i|z)\right)
\end{aligned}
$$

Rewriting the lower bound...

$$
\begin{aligned}
L^v &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(x|z)p(z)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z)}{q(z|x)} \right) + \sum_z q(z|x) \log p(x|z) \\
&= -D_{KL}\left(q(z|x)||p(z)\right) + E_{q(z|x)}\left(\log p(x|z)\right) \\
\text{for } x_i\ldots \quad &= -D_{KL}\left(q(z|x_i)||p(z)\right) + E_{q(z|x_i)}\left(\log p(x_i|z)\right)
\end{aligned}
$$

- Second term: Reconstruction loss; equals $\log(1)$ if x_i is perfectly reconstructed from z

Rewriting the lower bound...

$$
\begin{aligned}
L^v &= \sum_z q(z|x) \log \left(\frac{p(z,x)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(x|z)p(z)}{q(z|x)} \right) \\
&= \sum_z q(z|x) \log \left(\frac{p(z)}{q(z|x)} \right) + \sum_z q(z|x) \log p(x|z) \\
&= -D_{KL}\left(q(z|x)||p(z)\right) + E_{q(z|x)}\left(\log p(x|z)\right) \\
\text{for } x_i\ldots \quad &= -D_{KL}\left(q(z|x_i)||p(z)\right) + E_{q(z|x_i)}\left(\log p(x_i|z)\right)
\end{aligned}
$$

- Second term: Reconstruction loss; equals $\log(1)$ if x_i is perfectly reconstructed from z
- First term: Regularizer; prior $p(z)$ is usually taken to be $\mathcal{N}(0, \mathbf{I})$

Computing the lower bound for a training example

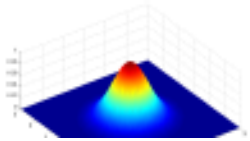
Example $x^{(i)}$



$q_\phi(z|x^{(i)})$



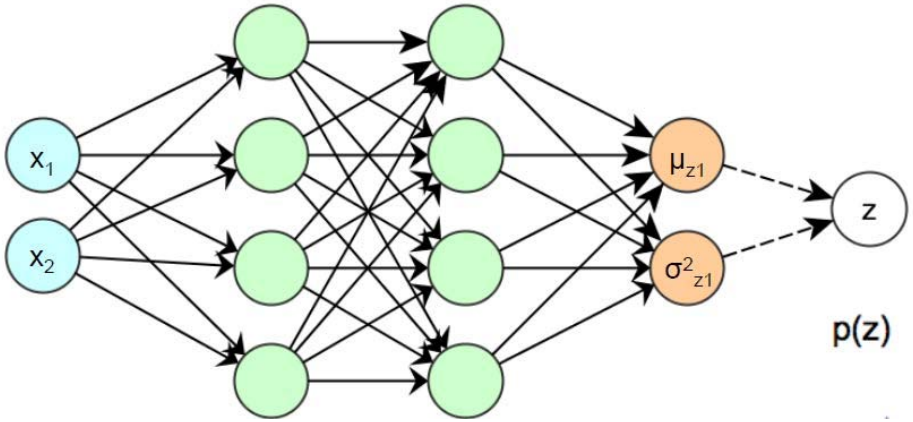
$p_\theta(x^{(i)}|z)$



[Dürr 2016]

Computing the lower bound for a training example

KL divergence term:

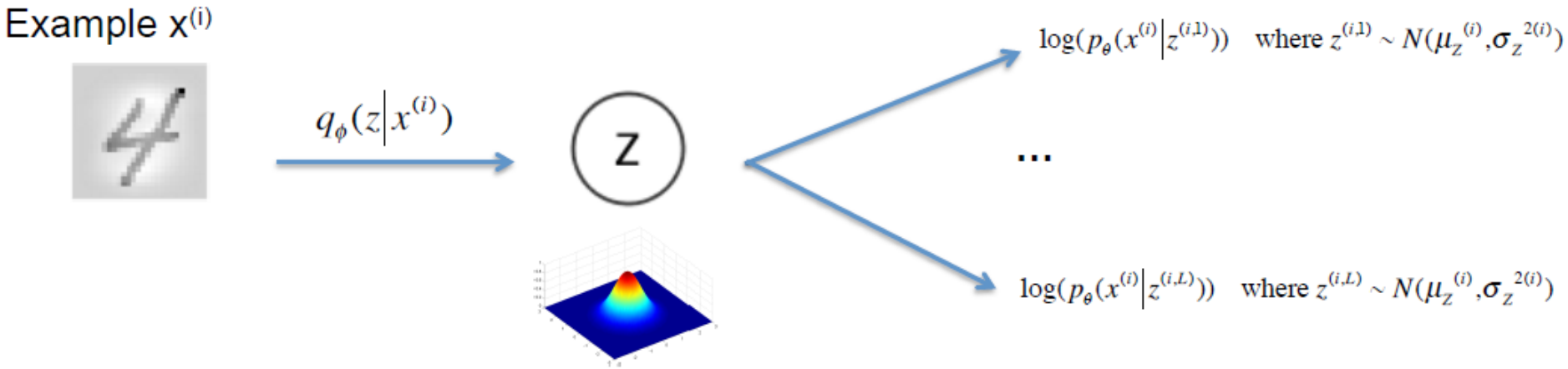


The KL divergence has a closed form when $p(z) = \mathcal{N}(0, \mathbf{I})$ and $q(z|x)$ is also Gaussian ($J =$ dimensionality of z):

$$-D_{KL}\left(q(z|x_i)||p(z)\right) = \frac{1}{2}\sum_{j=1}^{J} 1 + \log\left(\sigma^2_{z_{i,j}}\right) - \mu^2_{z_{i,j}} - \sigma^2_{z_{i,j}}$$

Computing the lower bound for a training example

Reconstruction term:



[Dürr 2016]

Approximate the expectation by sampling L samples from $q(z|x_i)$:

$$\begin{aligned} L^v &= -D_{KL}\left(q(z|x)||p(z)\right) + E_{q(z|x)}\left(\log p(x|z)\right) \\ &\approx -D_{KL}\left(q(z|x)||p(z)\right) + \frac{1}{L}\sum_{l=1}^{L}\left(\log p(x_i|z_i^l)\right) \end{aligned}$$

Often just use $L = 1$

Computing the lower bound for a training example

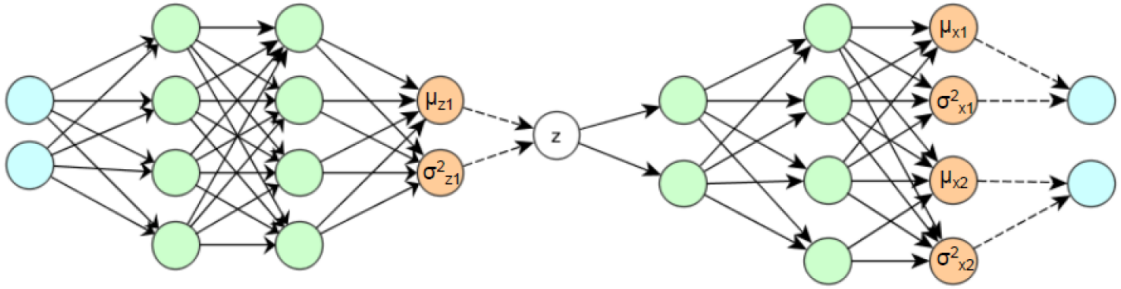
Next to compute the reconstruction term, we need to pick a form for $p(x|z)$

- Common choice: $p(x|z)$ is Gaussian with D (uncorrelated) dimensions
- If $L = 1$, then:

$$(\log p(x_i|z_i)) = \sum_{j=1}^{D} \frac{1}{2} \log \sigma_{x_j}^2 + \frac{\left(x_{i,j} - \mu_{x_j}\right)^2}{2\sigma_{x_j}^2}$$

- Another common choice: $\sigma_{x_j} =$ constant
- Then this is just a least squares reconstruction loss!

Putting it all together...

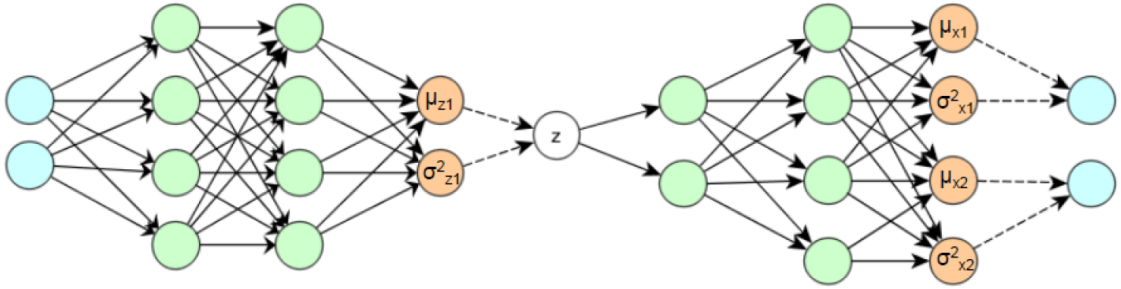


[Dürr 2016]

We will optimize the lower bound on the log likelihood. For x_i that is

$$L^v = -D_{KL}\left(q(z|x_i)||p(z)\right) + E_{q(z|x_i)}\left(\log p(x_i|z)\right)$$

Putting it all together...



[Dürr 2016]

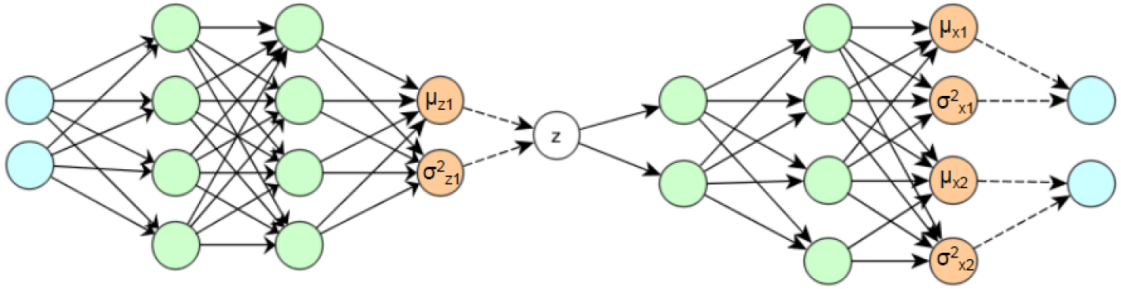
We will optimize the lower bound on the log likelihood. For x_i that is

$$L^v = -D_{KL}\left(q(z|x_i)||p(z)\right) + E_{q(z|x_i)}\left(\log p(x_i|z)\right)$$

Second term: Reconstruction loss estimated from single sample of z

$$\log p(x_i|z_i) = \sum_{j=1}^{D} \frac{1}{2} \log \sigma_{x_j}^2 + \left(x_{i,j} - \mu_{x_j}\right)^2 / (2\sigma_{x_j}^2)$$

Putting it all together...



[Dürr 2016]

We will optimize the lower bound on the log likelihood. For x_i that is

$$L^v = -D_{KL}\left(q(z|x_i)||p(z)\right) + E_{q(z|x_i)}\left(\log p(x_i|z)\right)$$

Second term: Reconstruction loss estimated from single sample of z

$$\log p(x_i|z_i) = \sum_{j=1}^{D} \frac{1}{2}\log \sigma_{x_j}^2 + \left(x_{i,j} - \mu_{x_j}\right)^2 / (2\sigma_{x_j}^2)$$

First term: Regularizer (with $p(z)$ taken to be $\mathcal{N}(0, \mathbf{I})$)

$$-D_{KL}\left(q(z|x_i)||p(z)\right) = \frac{1}{2}\sum_{j=1}^{J} 1 + \log\left(\sigma_{z_{i,j}}^2\right) - \mu_{z_{i,j}}^2 - \sigma_{z_{i,j}}^2$$

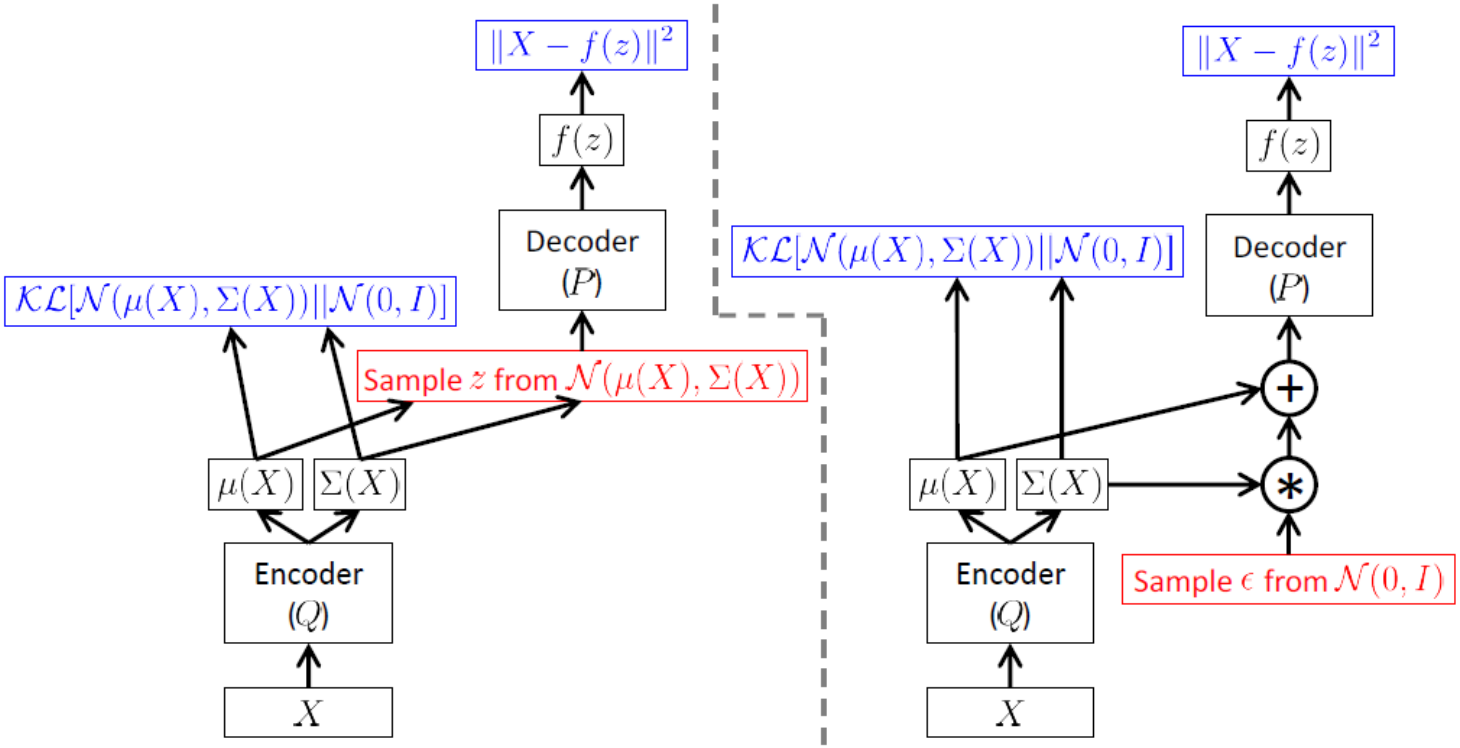
Can learn all parameters via backpropagation... or can we?

Reparameterization trick

- Not so fast... can't backpropagate through the random sampling
- Instead we will use the "reparameterization trick"
- Write $z = \mu_z + \sigma_z \times \epsilon$

Reparameterization trick

- Not so fast... can't backpropagate through the random sampling
- Instead we will use the "reparameterization trick"
- Write $z = \mu_z + \sigma_z \times \epsilon$



[Doersch 2016]

Relationship between AEs, denoising AEs, VAEs

Autoencoder:

- Pass input through encoder, then decoder
- Measure squared reconstruction loss between input and output

Relationship between AEs, denoising AEs, VAEs

Autoencoder:

- Pass input through encoder, then decoder
- Measure squared reconstruction loss between input and output

Denoising autoencoder:

- Add noise to input, pass through encoder, then decoder
- Measure squared reconstruction loss between (original) input and output

Relationship between AEs, denoising AEs, VAEs

Autoencoder:

- Pass input through encoder, then decoder
- Measure squared reconstruction loss between input and output

Denoising autoencoder:

- Add noise to input, pass through encoder, then decoder
- Measure squared reconstruction loss between (original) input and output

Variational autoencoder (with spherical Gaussian distributions):

- Pass input through encoder, add noise, then pass to decoder
- Measure squared reconstruction loss between (original) input and output, plus a KL divergence regularizer

VAE reconstruction examples, circa 2015

Input

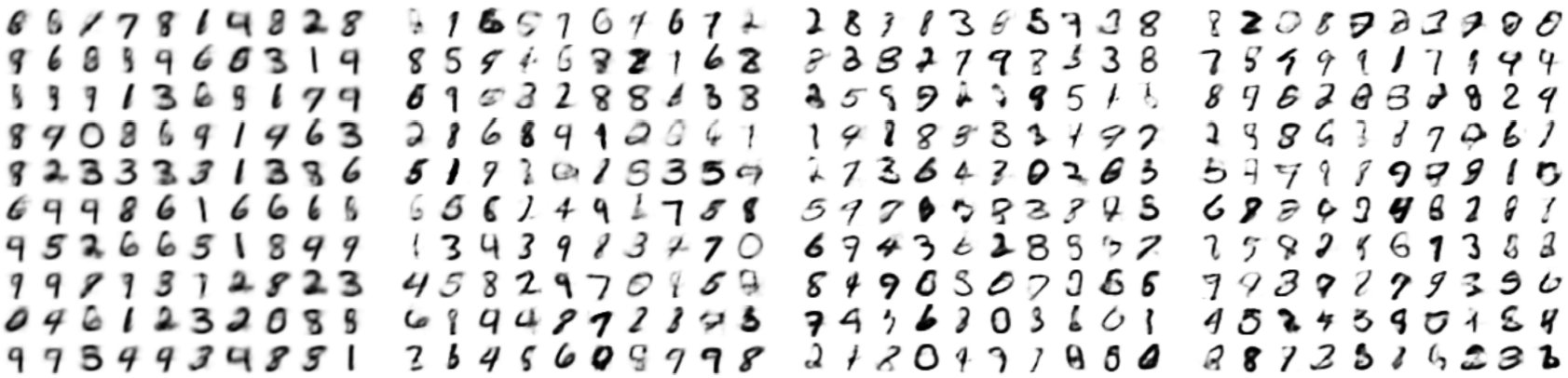


VAE reconstruction



[http://torch.ch/blog/2015/11/13/gan.html]

VAE reconstruction examples, circa 2015 (2)

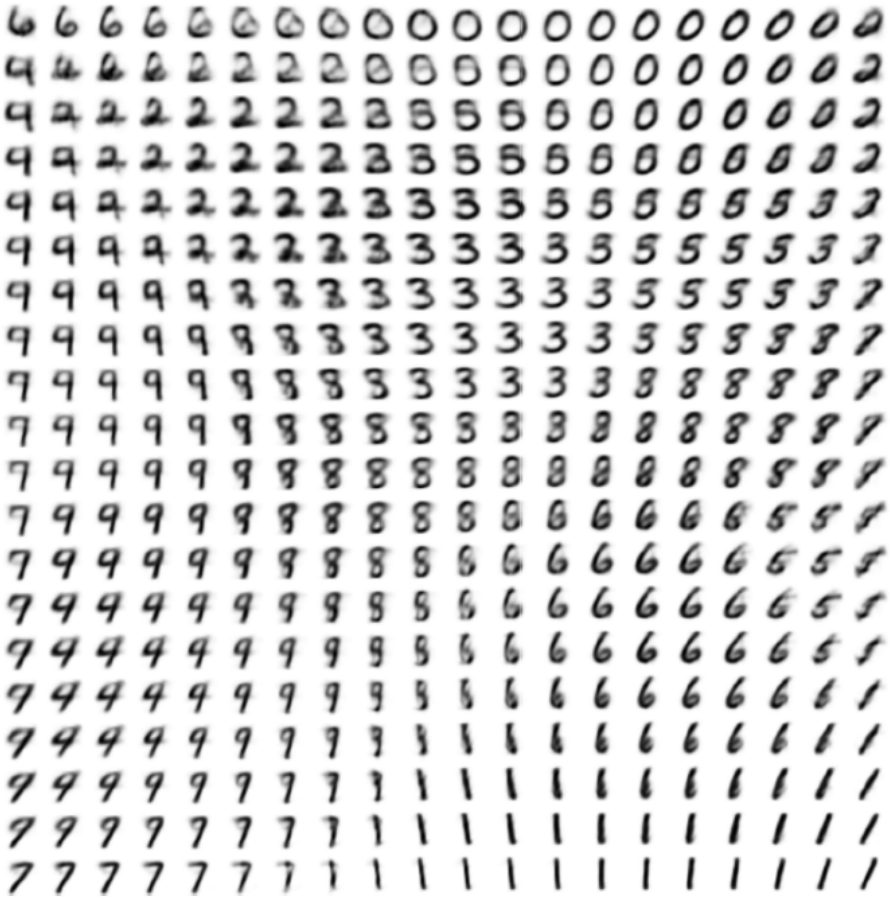


[Kingma & Welling 2014]

What do learned VAE dimensions represent?



(a) Learned Frey Face manifold



(b) Learned MNIST manifold

[Kingma & Welling 2014]

What should learned representations represent?

Interpretable dimensions of continuous variation

- Images: facial expression, digit writing style, ...
- Music: pitch, timbre, ...
- Speech audio: speaker characteristics, emotion, ...
- Text: shades of meaning, sentiment, ...

What should learned representations represent?

Or, everything that is not "noise"

- Goal is not necessarily interpretability
- Goal may be compression
- Goal may be to serve as input for better supervised learning

Common scenario: lots of unlabeled data, small amount of labeled data

- On HW1, HW2: learn representation on unlabeled data, use it on labeled data
- Often, learned representation is further tuned on labeled data with task-specific loss

What should learned representations represent?

Or, interpretable discrete dimensions

- Images: digit identity, face identity, object type, ...
- Music: instrument identity, discrete pitch, ...
- Speech audio: phoneme, word, language, ...
- Text: part of speech, phrase (constituent) type, topic, ...

This leads us to clustering...

Clustering

Clustering

- Many data sets naturally consist of multiple natural subsets, e.g. multiple digits, object classes in images, topics, phonemes in speech, ...
- Clustering = the problem of identifying these subsets and assigning data points to the correct one
- One of the key tasks in unsupervised learning and data analysis
- **Warning:** Almost no theoretical guarantees ☹. Treat all clustering results with some skepticism.

Clustering application: Vector quantization (VQ)

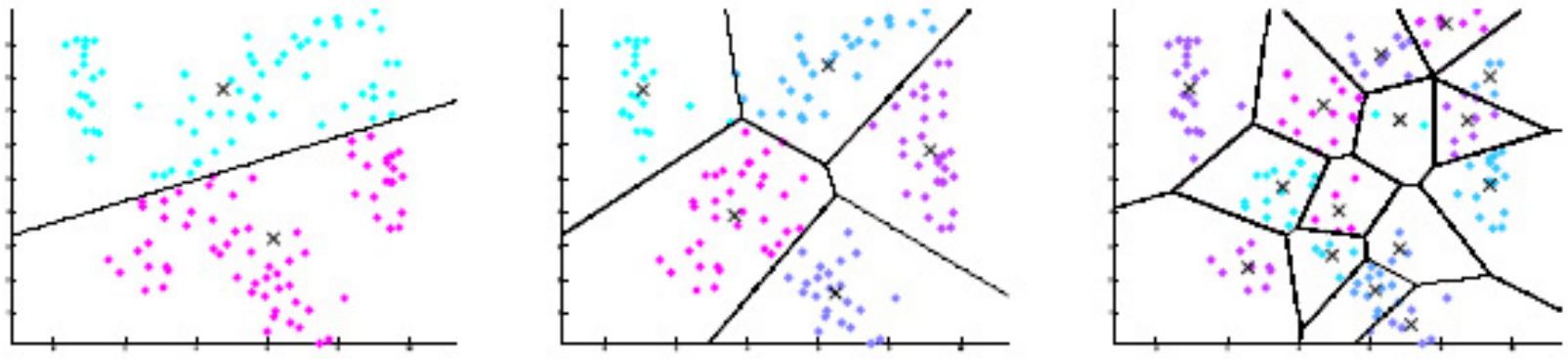
VQ is a clustering-based discrete representation

- Given feature vectors = vector of real values per data sample
- Quantization = conversion of feature vectors to discrete symbols using a "codebook"
- For some applications, compact representation may be important
- Sometimes, the *goal* is compactness (compression, coding)
- Quantization incurs some loss of information, however

Clustering application: Vector quantization (VQ)

VQ is a clustering-based discrete representation

- Given feature vectors = vector of real values per data sample
- Quantization = conversion of feature vectors to discrete symbols using a "codebook"
- For some applications, compact representation may be important
- Sometimes, the *goal* is compactness (compression, coding)
- Quantization incurs some loss of information, however
- Example VQ codebooks for 2 dimensional vectors:

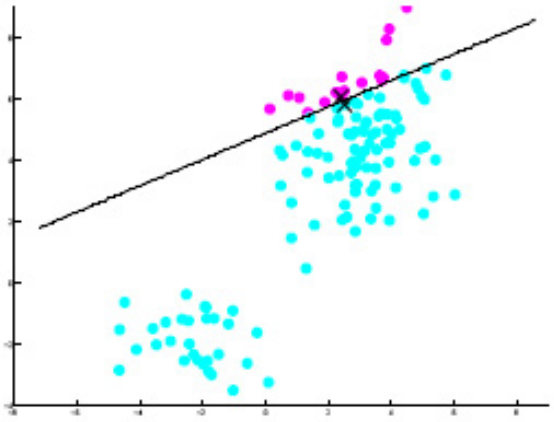


- Whenever need to represent $\mathbf{x}$, use instead the closest *codeword*

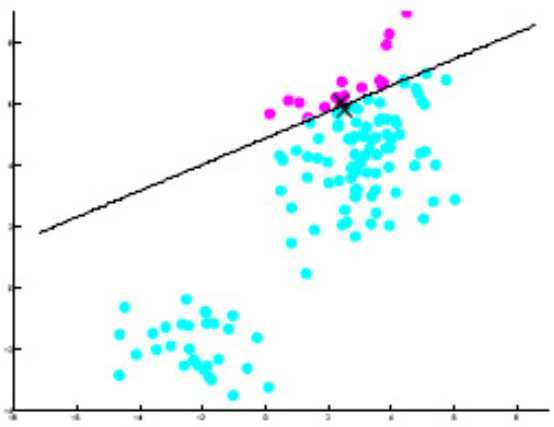
K-means clustering

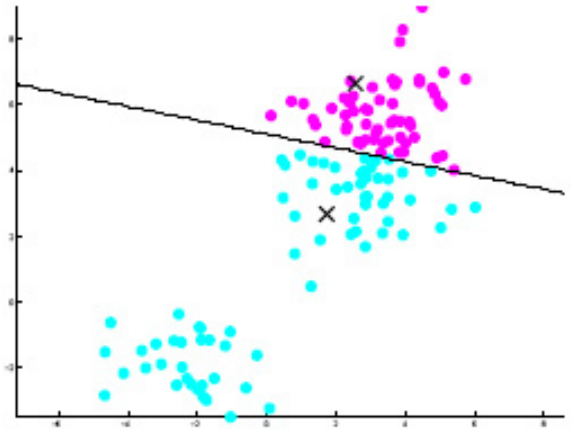
- Simplest VQ approach: k-means clustering
- Algorithm: given number of codewords (clusters) k and a set of examples,
  1. Initialize k means $\mu_1, \ldots, \mu_k$ to random locations
  2. Assign each example $\mathbf{x}_i$ to the closest mean: $y_i = \text{argmin}_c ||\mathbf{x}_i - \mu_c||$
  3. Reestimate each mean based only on examples assigned to it: $\mu_c = \frac{1}{N_c} \sum_{y_i=c} \mathbf{x}_i$, where $N_c = |\{\mathbf{x}_i : y_i = c\}|$
  4. Repeat 2 and 3 until there are no changes in assignment

K-means example with $k = 2$

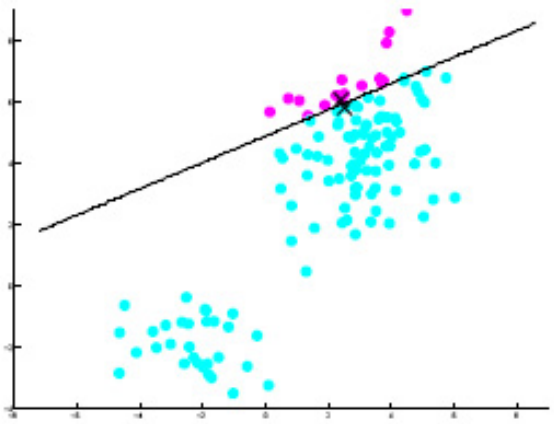


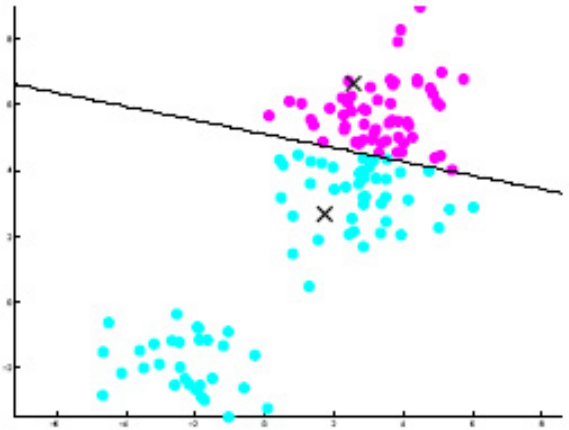
K-means example with $k = 2$

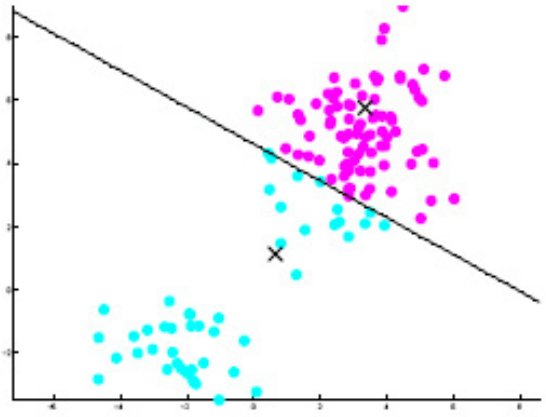




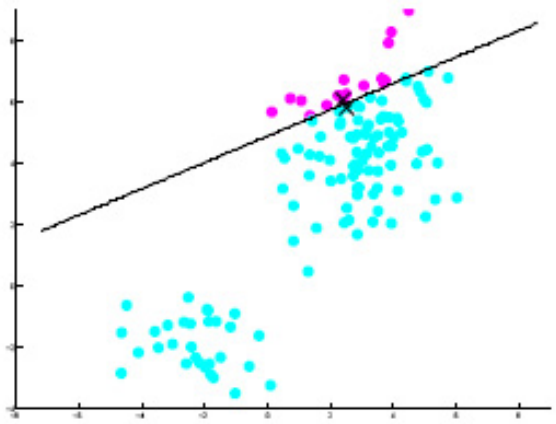
K-means example with $k = 2$

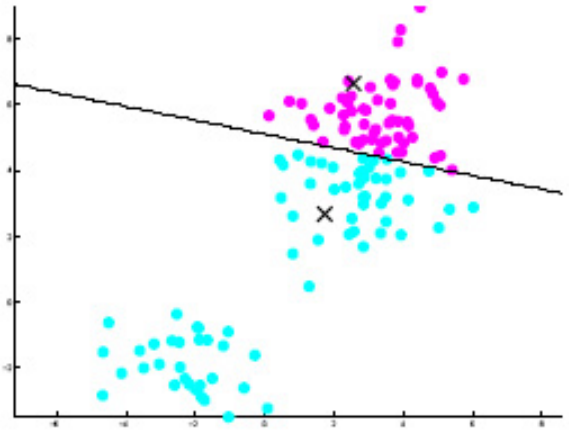


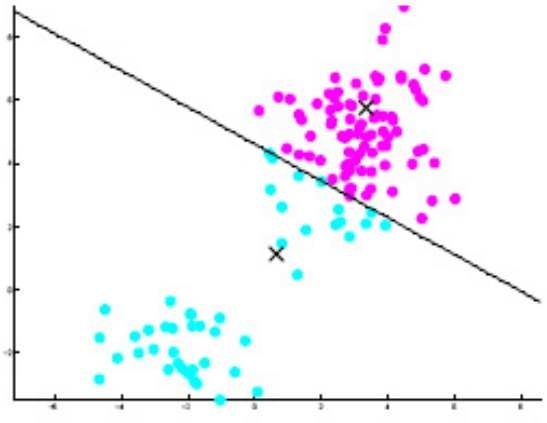


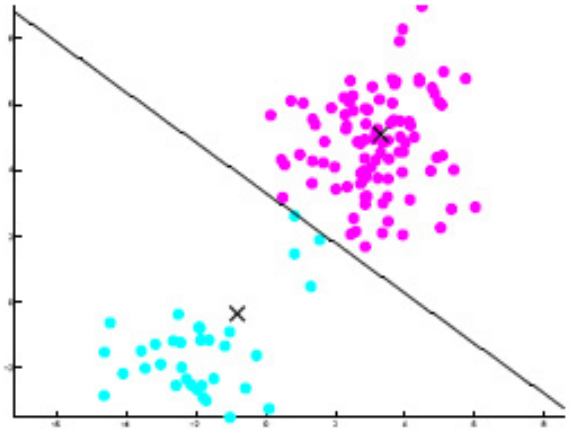


K-means example with $k = 2$

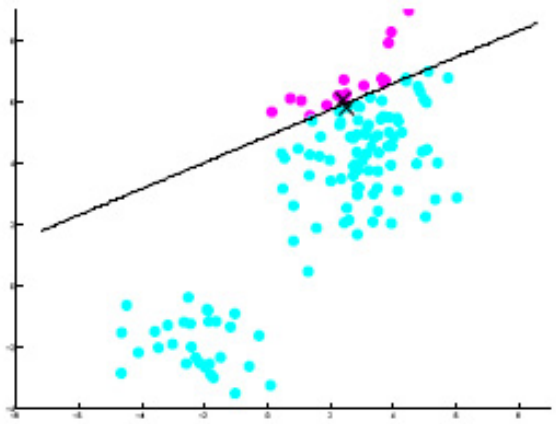


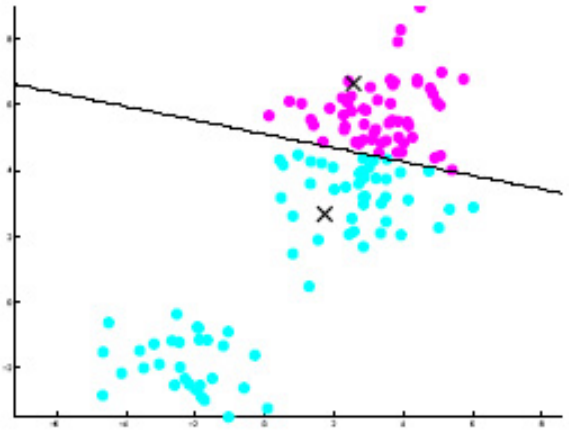


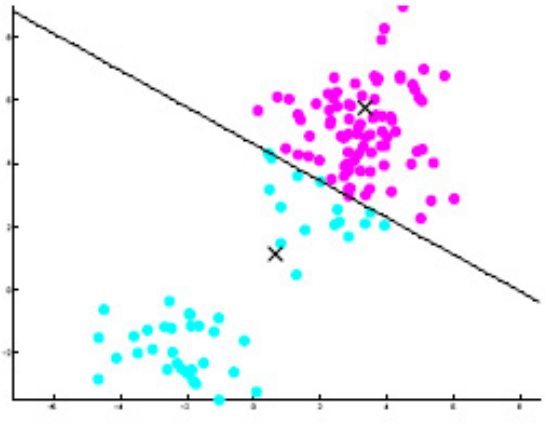


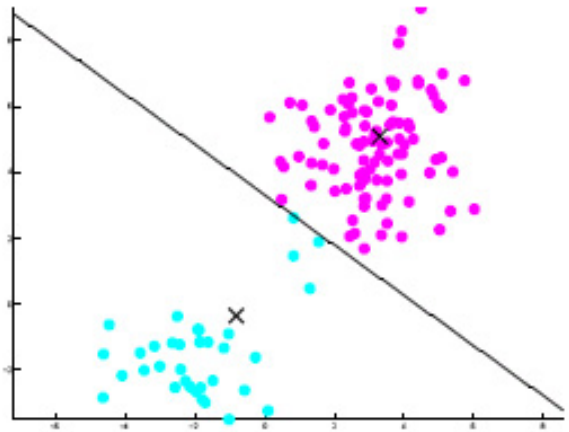


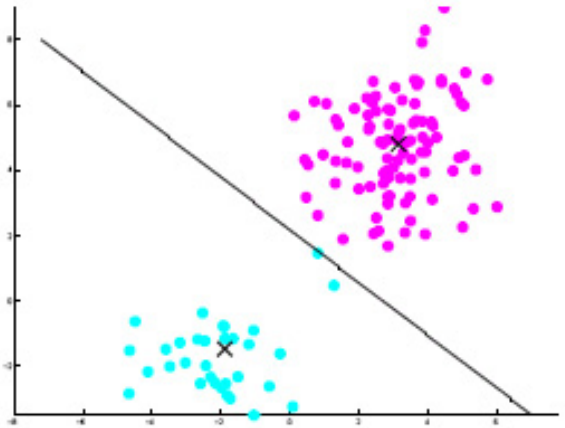
K-means example with $k = 2$



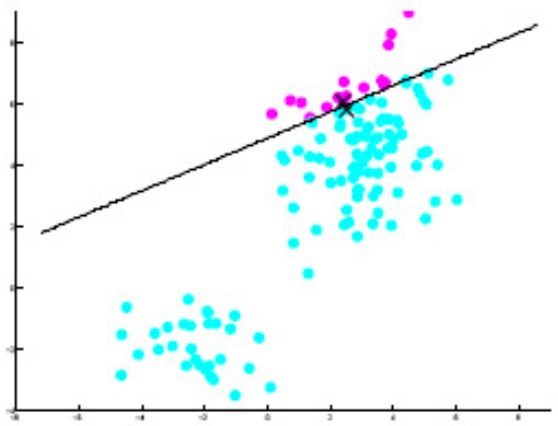
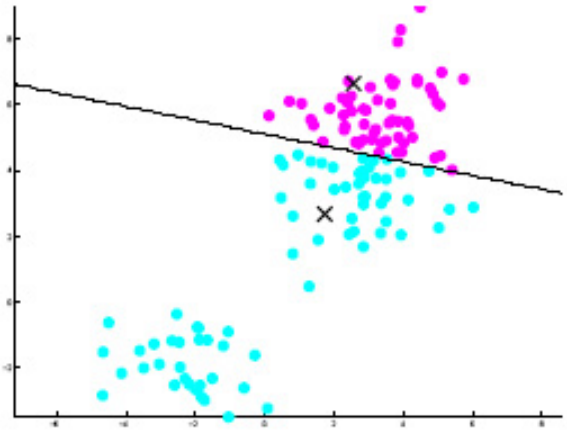
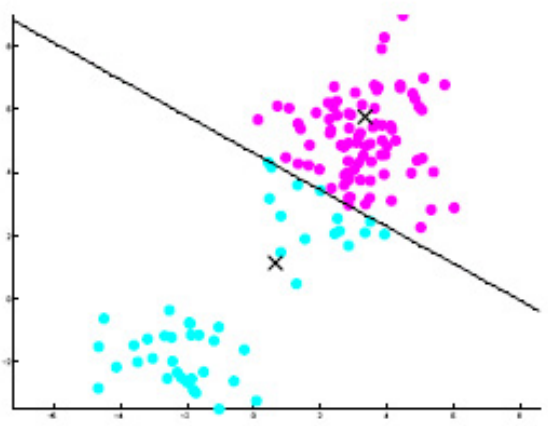
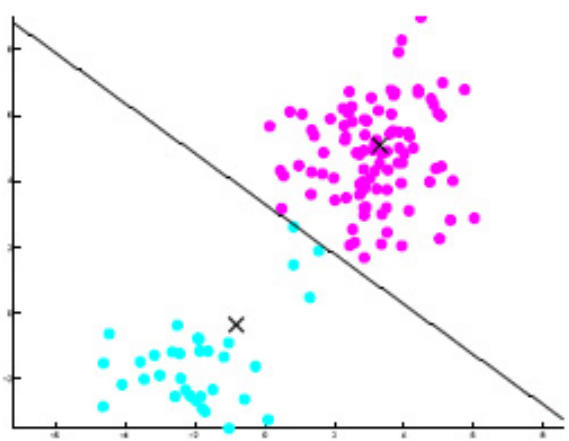
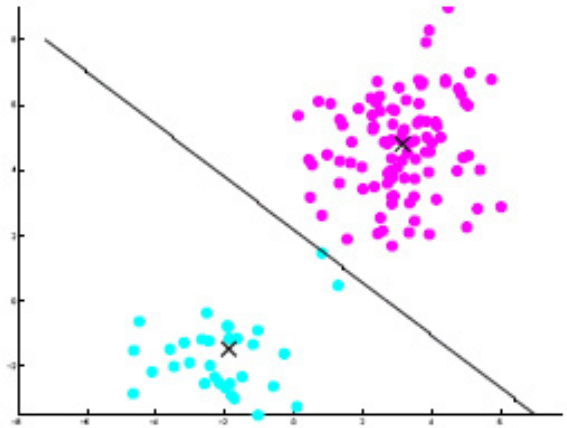
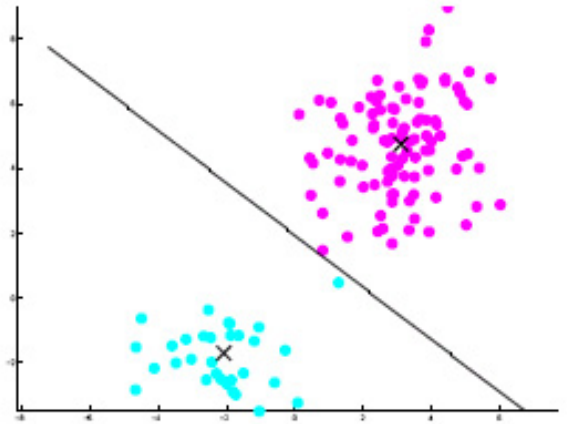








K-means example with $k = 2$

K-means properties

- Total distortion $D = \sum_{c=1}^{k} \sum_{\mathbf{x} \in \text{cluster } c} ||\mathbf{x} - \mu_c||^2$
- D decreases from one iteration to the next
- K-means converges to a local, but not necessarily global, minimum of D
- Initialization can affect final result
- Heuristic remedy: Run k-means with several initializations; keep the result with lowest D

K-means properties

- Total distortion $D = \sum_{c=1}^{k} \sum_{\mathbf{x} \in \text{cluster } c} ||\mathbf{x} - \mu_c||^2$
- D decreases from one iteration to the next
- K-means converges to a local, but not necessarily global, minimum of D
- Initialization can affect final result
- Heuristic remedy: Run k-means with several initializations; keep the result with lowest D
- Choice of distance measure interacts with cluster shape:



VQ example: Image segmentation/compression

